

Tyr-Crypto Computational Handbook

Volume 0: Foundations for reading cryptographic calculations

Revised teaching edition

No skipped conversions. No hidden arithmetic. Four worked examples for every method.

Blue	new input, public data, or a plain-language explanation
Purple	a rule or fixed mathematical procedure
Green	worked calculation
Orange	exercise for you
Red	mistake, danger, or security warning
Teal	checkpoint before moving on

Repository basis: [siriuslee69/Tyr-Crypto](#). This book teaches toy-sized calculations first and then connects them to the real implementation. It is educational material, not a claim that custom cryptographic code is safe for production.

Contents

Contents	1
1 How to use this volume	4
1.1 A low-friction study routine	4
1.2 Terms that will appear repeatedly	5
1.3 Where these calculations appear in Tyr-Crypto	6
2 Number systems: decimal, binary, and hexadecimal	7
2.1 Read binary as an ordinary number	8
2.2 Convert decimal to binary	9
2.3 Learn the hexadecimal digit map	11
2.4 Convert hexadecimal to binary	13
2.5 Convert binary to hexadecimal	14
2.6 Convert hexadecimal to decimal	15
2.7 Exercises: number systems	16
3 Bit operations: NOT, AND, OR, XOR, shifts, and rotations	17
3.1 NOT: flip every bit	18
3.2 AND: keep only shared 1-bits	19
3.3 OR: combine 1-bits	20
3.4 XOR: mark differences	21
3.5 Left shift: move bits toward larger place values	22
3.6 Right shift: move bits toward smaller place values	23
3.7 Rotation: move bits around a circle	24
3.8 Exercises: bit operations	25
4 Modular arithmetic: controlled wrap-around	26
4.1 Reduce a number modulo q	27
4.2 Add and subtract modulo q	28
4.3 Multiply modulo q	29
4.4 Compute powers modulo q	30
4.5 Find a modular inverse by search	31
4.6 Find an inverse with the extended Euclidean algorithm	32
4.7 Exercises: modular arithmetic	33
5 Polynomials as coefficient arrays	34
5.1 Translate between polynomial notation and arrays	35
5.2 Add and subtract polynomials	36
5.3 Multiply a polynomial by a scalar	37
5.4 Multiply polynomials by convolution	38
5.5 Reduce in the ring $x^n + 1$	39
5.6 Exercises: polynomial arithmetic	40

6 Vectors and matrices: repeated dot products	41
6.1 Compute a dot product	42
6.2 Multiply a matrix by a vector	43
6.3 Matrix-vector multiplication modulo q	44
6.4 Exercises: vectors and matrices	45
7 Centering, rounding, compression, and small noise	46
7.1 Choose a centered representative	47
7.2 Round to the nearest integer	48
7.3 Compress a residue to d bits	49
7.4 Decompress a d -bit value	50
7.5 Add small noise and recover a coarse value	51
7.6 Exercises: rounding and noise	52
8 States, permutations, sponges, and streams	53
8.1 Apply a toy state permutation	54
8.2 Absorb and squeeze in a toy sponge	55
8.3 Encrypt and decrypt with a stream using XOR	56
8.4 Exercises: states and streams	57
9 Nim preview: from calculations to buffers	58
9.1 Variables and fixed-width integer types	59
9.2 Fixed arrays and indexing	60
9.3 Loops with public fixed bounds	61
9.4 Unsigned wrap-around and explicit rotation	62
9.5 Buffers, stack, heap, and secret lifetime	63
9.6 Exercises: reading tiny Nim fragments	64
10 Cumulative guided calculations	65
10.1 Cumulative A: byte mixing pipeline	66
10.2 Cumulative B: toy lattice pipeline	67
10.3 Cumulative C: toy sponge and stream	68
11 Compact solutions	69
11.1 Number-system solutions	70
11.2 Bit-operation solutions	71
11.3 Modular-arithmetic solutions	72
11.4 Polynomial solutions	73
11.5 Vector and matrix solutions	74
11.6 Rounding and noise solutions	75
11.7 State, sponge, and stream solutions	76
11.8 Nim-reading solutions	77
11.9 Cumulative solutions	78
12 What follows in the algorithm volumes	79

13 Primary references and repository links

80

1 How to use this volume

The promise of this book

You are not expected to know computer-number notation, bit operations, modular arithmetic, polynomial rings, or memory terminology in advance.

Every new method follows the same order: plain meaning -> exact rule -> tiny steps -> four worked examples -> common mistake -> checkpoint.

Exercises are grouped by difficulty. Later cumulative exercises combine old and new steps, but they include a scaffold so that you do not have to hold the entire algorithm in working memory at once.

1.1 A low-friction study routine

Pass 1: recognize

Read only the blue and purple boxes. Your goal is merely to recognize the symbols and the order of steps.

Pass 2: copy

Cover the green result column and copy each worked calculation by hand. Copying is allowed; it reduces working-memory load.

Pass 3: retrieve

Do the easiest orange exercises without looking. Mark the exact line where you became unsure.

Pass 4: connect

Do one cumulative exercise. Use the printed scaffold. Speed is irrelevant.

When to stop

Stop after one page if the symbols begin to blur together. Cryptographic calculations are repetitive; a short, accurate session is more useful than a long confused session.

1.2 Terms that will appear repeatedly

Term	Meaning without jargon	Concrete picture
bit	One yes/no position. It is written as 0 or 1.	one light switch
byte	A row of 8 bits. Example: 00101101.	eight switches in a row
word	A fixed-size group of bits that a CPU treats as one number, often 16, 32, or 64 bits.	one CPU-sized tray of switches
state	The complete working data currently being transformed. Think of a worksheet with several numbered boxes.	all trays on the desk
buffer	A reserved row of memory cells that temporarily stores bytes.	empty labeled storage boxes
coefficient	One number in a polynomial. In $3 + 2x + 5x^2$, the coefficients are 3, 2, and 5.	the number attached to one power of x
modulus	The wrap-around size. Modulo 7 means numbers wrap after 6 back to 0.	a clock face
nonce	A value intended to be used once with a key. It usually does not need to be secret, but reuse can be dangerous.	a ticket number that must not repeat
key	Secret or public data that selects a particular cryptographic transformation.	the special setting of a lock
digest	The fixed-size output of a hash function.	a compact fingerprint
tag	A short value used to detect modification or forgery.	a tamper-evident seal
permutation	A reversible rearrangement/transformation of a fixed-size state.	a reversible mixing machine

1.3 Where these calculations appear in Tyr-Crypto

Family	Calculations you will later recognize	Repository area
ChaCha20 / XChaCha20	32-bit addition, XOR, rotation	symmetric/chacha
AES-CTR	bytes, XOR, finite-field byte transforms, counters	symmetric/aes
SHA3 / SHAKE	XOR, rotations, state lanes, sponge absorb/squeeze	symmetric/sha3
BLAKE3	32-bit addition, XOR, rotations, tree combination	symmetric/blake3
Gimli	32-bit rotations, XOR, state permutation	symmetric/gimli
Poly1305	large-integer arithmetic modulo 2^{130-5}	symmetric/poly1305
Argon2 / custom KDF	buffers, repeated memory mixing, indexing, lifetimes	symmetric/argon2 and kdf.nim
X25519 / Ed25519	finite-field arithmetic, fixed-length bit processing	asymmetric/none_pq
Kyber / Dilithium	polynomials, modular arithmetic, NTT, rounding	asymmetric/pq/kyber and dilithium
SABER	polynomial multiplication and power-of-two rounding	asymmetric/pq/saber
NTRU	polynomial rings, inverses, fixed-weight coefficients	asymmetric/pq/ntru
FrodoKEM	large matrix products, noise, encode/decode	asymmetric/pq/frodo
BIKE / McEliece	binary polynomials, XOR, code-based decoding	asymmetric/pq/bike and mceliece
Falcon	polynomials, FFT-style transforms, Gaussian sampling	asymmetric/pq/falcon
SPHINCS+	hash chains, trees, indexes, many fixed-size buffers	asymmetric/pq/sphincs

Important

The real algorithms use much larger numbers and arrays. The toy examples preserve the shape of the calculation while keeping every step writable by hand.

2 Number systems: decimal, binary, and hexadecimal

Why this chapter exists

Cryptographic source code constantly switches between ordinary decimal numbers, binary bit patterns, and hexadecimal byte notation. The conversion is not obvious if nobody has shown the place values. This chapter starts from zero.

One fact to keep visible

Binary uses powers of 2. Hexadecimal uses powers of 16. One hexadecimal digit always corresponds to exactly four binary bits.

2.1 Read binary as an ordinary number

On this page	After this page
Binary place values and why the rightmost bit means 1.	Convert a binary row into decimal without guessing.

Plain-language meaning

A binary number is not a mysterious code. Each position has a value. Starting from the right, the values are 1, 2, 4, 8, 16, 32, 64, 128, A bit of 1 means "include this value". A bit of 0 means "do not include it".

The rule

... 128 64 32 16 8 4 2 1

Write the place values above the bits, multiply each place value by its bit, then add.

Do it in these small steps

- Step 1. Write powers of 2 above the bit positions, beginning with 1 on the far right.
- Step 2. Cross out every place whose bit is 0.
- Step 3. Add the place values whose bit is 1.
- Step 4. Write the decimal answer and keep the original binary width visible.

Four fully worked examples

Example	Start	Every visible step	Answer
1	1011	8 4 2 1 $1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$ $= 8 + 0 + 2 + 1$	11
2	10010	16 8 4 2 1 $1 \times 16 + 0 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1$	18
3	111111	$32 + 16 + 8 + 4 + 2 + 1$	63
4	01010101	$0 \times 128 + 1 \times 64 + 0 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$	85

Common mistake

Do not read 1011 as "one thousand eleven". The base is different, so the place values are different.

Stop and check

Can you point to the bit worth 8? Can you explain why a leading 0 changes the width but not the numerical value?

2.2 Convert decimal to binary

On this page	After this page
Building a binary row from powers of 2.	Convert a normal decimal number into binary by a fixed checklist.

Plain-language meaning

You choose which powers of 2 are needed to make the number. Start with the largest power that is not too large. Subtract it, write 1 in that position, and continue downward. Write 0 for every skipped power.

The rule

Largest fitting power first. At each power: Does it still fit into the remaining amount? Yes -> write 1 and subtract. No -> write 0.

Do it in these small steps

Step 1. List powers of 2 from the largest useful one down to 1.
 Step 2. Start with the full decimal number as the remainder.
 Step 3. For each power, write 1 and subtract when it fits; otherwise write 0.
 Step 4. When the remainder reaches 0, still write zeros for any remaining lower places.

Four fully worked examples

Example	Start	Every visible step	Answer
1	13	Powers: 8, 4, 2, 1 $13-8=5 \rightarrow 1$ $5-4=1 \rightarrow 1$ 2 does not fit $\rightarrow 0$ $1-1=0 \rightarrow 1$	1101
2	20	Powers: 16, 8, 4, 2, 1 $20-16=4 \rightarrow 1$ 8 no $\rightarrow 0$ $4-4=0 \rightarrow 1$ then 0,0	10100
3	37	32 fits $\rightarrow 1$, remainder 5 16,8 do not fit $\rightarrow 0,0$ 4 fits $\rightarrow 1$, remainder 1 2 no $\rightarrow 0$; 1 fits $\rightarrow 1$	100101
4	156	128 fits, remainder 28 64,32 no 16 fits, rem.12 8 fits, rem.4 4 fits, rem.0 2,1 no	10011100

Common mistake

Do not omit a zero in the middle. In 20, the 8-place is empty, so the binary form must visibly contain a 0 there.

Stop and check

Convert your result back to decimal. If it does not return the starting number, one place value was skipped.

2.3 Learn the hexadecimal digit map

On this page	After this page
Why hexadecimal exists and what A-F mean.	Read one hexadecimal digit and replace it with four bits.

Plain-language meaning

Hexadecimal is a compact way to write binary. Decimal has ten single digits: 0 through 9. Hexadecimal needs sixteen single digits, so it continues with A, B, C, D, E, F for decimal 10, 11, 12, 13, 14, 15.

Hex	Decimal	Four bits
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Four fully worked lookups

Example	Start	Every visible step	Answer
1	Hex 7	Find row 7. Decimal 7 uses 4+2+1.	0111
2	Hex A	A means decimal 10. 10 = 8+2.	1010
3	Hex C	C means decimal 12. 12 = 8+4.	1100
4	Hex F	F means decimal 15. 15 = 8+4+2+1.	1111

Common mistake

The letter O is not a hexadecimal digit. The digit is zero: 0. Also, A means ten only when the number is explicitly hexadecimal.

Stop and check

Cover the bit column. Can you reconstruct A, C, E, and F using 8-4-2-1?

2.4 Convert hexadecimal to binary

On this page	After this page
Replacing each hex digit independently with four bits.	Convert an entire hexadecimal byte or word into binary.

Plain-language meaning

Do not convert the whole number at once. Treat every hexadecimal digit as a separate four-bit tile. Replace each tile using the map, then place the tiles next to one another.

The rule

One hex digit = four bits. Keep all four bits, including leading zeros inside a tile.

Do it in these small steps

Step 1. Split the hexadecimal number into individual digits.
 Step 2. Look up the four-bit pattern for each digit.
 Step 3. Write the four-bit groups in the same left-to-right order.
 Step 4. Keep spaces between groups until the final check.

Four fully worked examples

Example	Start	Every visible step	Answer
1	0x2F	2 -> 0010 F -> 1111 Join: 0010 1111	00101111
2	0xA4	A -> 1010 4 -> 0100	10100100
3	0x09C	0 -> 0000 9 -> 1001 C -> 1100	000010011100
4	0xB7E1	B -> 1011; 7 -> 0111; E -> 1110; 1 -> 0001	101101111110 0001

Common mistake

Do not write 4 as 100 when converting from hex. It must occupy its full four-bit tile: 0100.

Stop and check

Does your binary result contain exactly four bits for every hex digit? Two hex digits must produce eight bits.

2.5 Convert binary to hexadecimal

On this page	After this page
Packing bits into groups of four.	Compress a long binary row into hex notation.

Plain-language meaning

This is the reverse of the previous method. Starting from the right, draw boundaries every four bits. If the leftmost group is short, add zeros only on its left. Convert each group using the hex map.

The rule

Group from the right-hand side: Pad only the leftmost group with leading zeros.

Do it in these small steps

Step 1. Begin at the rightmost bit.
 Step 2. Make groups of exactly four bits moving left.
 Step 3. Pad the leftmost group on the left if needed.
 Step 4. Convert each four-bit group to one hex digit.

Four fully worked examples

Example	Start	Every visible step	Answer
1	00101111	0010 -> 2 1111 -> F	0x2F
2	10100100	1010 -> A 0100 -> 4	0xA4
3	110101	Group from right: 11 0101 Pad: 0011 0101 0011 -> 3; 0101 -> 5	0x35
4	1110001110011	1 1100 0111 0011 Pad: 0001 1100 0111 0011	0x1C73

Common mistake

Grouping from the left gives the wrong place values when the bit count is not a multiple of four.

Stop and check

Expand your hexadecimal answer back into four-bit groups. It should match the padded binary row exactly.

2.6 Convert hexadecimal to decimal

On this page	After this page
Using powers of 16.	Find the ordinary numerical value of a hexadecimal number.

Plain-language meaning

Hexadecimal place values are 1, 16, 256, 4096, ... from right to left. Replace A-F with 10-15, multiply by the place value, and add.

The rule

For two hex digits XY: value = $X \times 16 + Y$. For three digits: $X \times 256 + Y \times 16 + Z$.

Do it in these small steps

Step 1. Write 1 under the rightmost digit, then 16, 256, 4096 moving left.
 Step 2. Replace A-F with decimal 10-15.
 Step 3. Multiply each digit value by its place value.
 Step 4. Add all products.

Four fully worked examples

Example	Start	Every visible step	Answer
1	0x2F	$2 \times 16 + 15 \times 1 = 32 + 15$	47
2	0xA4	$10 \times 16 + 4 = 160 + 4$	164
3	0x10B	$1 \times 256 + 0 \times 16 + 11$	267
4	0x3E8	$3 \times 256 + 14 \times 16 + 8$ $= 768 + 224 + 8$	1000

Common mistake

Do not treat 0x10 as decimal ten. Hex 0x10 is one group of sixteen plus zero ones, so it is decimal 16.

Stop and check

Check 0xFF: $15 \times 16 + 15 = 255$. This boundary appears constantly in byte-oriented code.

2.7 Exercises: number systems

How to use this exercise page

Do one group at a time. The first group is recognition, the last group combines conversions. Write place values above the digits rather than doing it mentally.

Level A - read binary

- B1. Convert 101 to decimal.
- B2. Convert 100000 to decimal.
- B3. Convert 01101100 to decimal.
- B4. Convert 11110001 to decimal.

Level B - build binary

- B5. Convert decimal 9 to binary.
- B6. Convert decimal 26 to binary.
- B7. Convert decimal 73 to an 8-bit binary row.
- B8. Convert decimal 255 to binary.

Level C - hex tiles

- B9. Convert 0x5D to binary.
- B10. Convert 0xC03 to binary.
- B11. Convert 01111110 to hex.
- B12. Convert 101101001 to hex.

Level D - combined

- B13. Convert 0x7B to decimal by powers of 16.
- B14. Convert decimal 210 to binary and then to hex.
- B15. Convert 0x123 to binary and decimal.
- B16. A byte is 11101010. Give its decimal and hex forms.

3 Bit operations: NOT, AND, OR, XOR, shifts, and rotations

What "bitwise" means

The operation is performed separately on each vertical bit column. Align equal-width rows first. The leftmost bit only interacts with the leftmost bit; it does not jump to another position.

Width must be stated

NOT, shifts, and rotations depend on the chosen width. This book writes "8-bit" or "4-bit" explicitly. Real code uses types such as `uint8`, `uint32`, or `uint64` to fix the width.

3.1 NOT: flip every bit

On this page	After this page
The one-input truth table and fixed-width inversion.	Invert a bit row without changing its width.

Plain-language meaning

NOT means "opposite" for each bit. Every 0 turns into 1 and every 1 turns into 0. The width stays unchanged.

Truth table for NOT

input a	NOT a
0	1
1	0

NOT flips one bit: 0 becomes 1, and 1 becomes 0.

The rule

For each position: 0 -> 1 and 1 -> 0.

Do it in these small steps

Step 1. Write the width above the row.
 Step 2. Copy the row one bit at a time.
 Step 3. Replace 0 with 1 and 1 with 0.
 Step 4. Count the result bits to confirm the width stayed fixed.

Four fully worked examples

Example	Start	Every visible step	Answer
1	4-bit 0000	0->1 four times	1111
2	4-bit 1010	1->0, 0->1, 1->0, 0->1	0101
3	8-bit 00110110	00 11 01 10 -> 11 00 10 01	11001001
4	8-bit 11110000	1111 -> 0000; 0000 -> 1111	00001111

Common mistake

NOT 0001 is not simply "-1" unless a signed-number convention and width are specified. At this stage, treat it only as bit flipping.

Stop and check

Apply NOT twice. You must return to the original row.

3.2 AND: keep only shared 1-bits

On this page	After this page
The two-input truth table and vertical alignment.	Calculate a bit mask with AND.

Plain-language meaning

AND keeps a 1 only where both rows already contain a 1. It is often used with a mask to keep selected bit positions and clear the rest.

Truth table for AND

a	b	a AND b
0	0	0
0	1	0
1	0	0
1	1	1

AND outputs 1 only when both input bits are 1. Think: "both conditions must be true."

The rule

Column by column: 1 AND 1 = 1. Every other pair gives 0.

Do it in these small steps

Step 1. Make both rows the same width.
 Step 2. Write one row directly under the other.
 Step 3. Use the truth table on each vertical pair.
 Step 4. Read the result row from left to right.

Four fully worked examples

Example	Start	Every visible step	Answer
1	1010 AND 1100	1&1=1; 0&1=0; 1&0=0; 0&0=0	1000
2	1111 AND 0101	The second row acts as the mask.	0101
3	00110110 AND 00001111	Upper four positions meet zeros; lower four are preserved.	00000110
4	0xD6 AND 0x3F	D6=11010110; 3F=00111111 AND each column	00010110 = 0x16

Common mistake

AND is not multiplication of the whole numbers. It only resembles multiplication for a single bit.

Stop and check

A AND all-ones should equal A. A AND all-zeros should equal all-zeros.

3.3 OR: combine 1-bits

On this page	After this page
The OR truth table.	Merge selected bit flags.

Plain-language meaning

OR places a 1 wherever either input row has a 1. It is often used to turn on options or combine independent flags.

Truth table for OR

a	b	a OR b
0	0	0
0	1	1
1	0	1
1	1	1

OR outputs 1 when at least one input bit is 1. Think: "one or both conditions may be true."

The rule

Column by column: only 0 OR 0 gives 0. Every pair containing at least one 1 gives 1.

Do it in these small steps

- Step 1. Align equal-width rows.
- Step 2. Inspect one vertical pair at a time.
- Step 3. Write 0 only when both bits are 0; otherwise write 1.
- Step 4. Check that every original 1 remains a 1 in the result.

Four fully worked examples

Example	Start	Every visible step	Answer
1	1010 OR 1100	1 1=1; 0 1=1; 1 0=1; 0 0=0	1110
2	0001 OR 0100	The separate 1-bits are combined.	0101
3	00110110 OR 00001111	Upper part stays 0011; lower part becomes 1111.	00111111
4	0x82 OR 0x35	82=10000010; 35=00110101	10110111 = 0xB7

Common mistake

In ordinary English, "or" sometimes means one but not both. Bitwise OR includes the case where both bits are 1.

Stop and check

A OR all-zeros should equal A. A OR all-ones should equal all-ones.

3.4 XOR: mark differences

On this page	After this page
The XOR truth table and its reversible property.	Calculate XOR and undo it with the same mask.

Plain-language meaning

XOR is pronounced "exclusive or". It returns 1 when the two input bits differ and 0 when they are equal. In stream ciphers, plaintext is XORed with a keystream. Applying the same keystream again restores the plaintext.

Truth table for XOR

a	b	a XOR b
0	0	0
0	1	1
1	0	1
1	1	0

XOR outputs 1 when the two bits are different. Equal bits produce 0.

The rule

Equal -> 0. Different -> 1. Also: $(A \text{ XOR } K) \text{ XOR } K = A$.

Do it in these small steps

Step 1. Align the two rows.
 Step 2. Compare each bit pair.
 Step 3. Write 0 for equal pairs and 1 for different pairs.
 Step 4. For a reversibility check, XOR the result with the second input again.

Four fully worked examples

Examp e	Start	Every visible step	Answer
1	1010 XOR 1100	$1 \wedge 1 = 0$; $0 \wedge 1 = 1$; $1 \wedge 0 = 1$; $0 \wedge 0 = 0$	0110
2	1111 XOR 0101	Different at positions 1 and 3 from the right.	1010
3	00110110 XOR 11110000	Compare all eight columns.	11000110
4	P=0x3A, K=0xC7	3A=00111010; C7=11000111 XOR -> 11111101	0xFD

Common mistake

XOR is not OR. In the 1,1 row, OR gives 1 but XOR gives 0.

Stop and check

A XOR A must be all-zeros. A XOR all-zeros must be A. These are fast error checks.

3.5 Left shift: move bits toward larger place values

On this page	After this page
Fixed-width logical left shift.	Shift an unsigned bit row left and identify discarded bits.

Plain-language meaning

A left shift moves every bit toward the higher-value side. Zeros enter from the right. Bits that leave the left edge are discarded when the width is fixed.

The rule

Within a fixed width: $a \ll k$ means move every bit k positions left, insert k zeros on the right, discard overflow on the left.

Do it in these small steps

- Step 1. Write the fixed width.
- Step 2. Move every bit left by the requested count.
- Step 3. Insert zeros in the newly empty right positions.
- Step 4. Cross out bits that moved beyond the left boundary.

Four fully worked examples

Example	Start	Every visible step	Answer
1	4-bit 0011 $\ll$ 1	0011 $\rightarrow$ 0110	0110
2	8-bit 00010110 $\ll$ 2	00010110 $\rightarrow$ 01011000	01011000
3	8-bit 10110001 $\ll$ 1	Leftmost 1 leaves the byte; add 0 on right.	01100010
4	8-bit 0x3D $\ll$ 3	3D=00111101 $\rightarrow$ 11101000	0xE8

Common mistake

Left shift often resembles multiplication by 2^k only when no important 1-bit is discarded.

Stop and check

Count how many zeros entered on the right. It must equal the shift count.

3.6 Right shift: move bits toward smaller place values

On this page	After this page
Logical right shift for unsigned values.	Shift a bit row right and distinguish it from rotation.

Plain-language meaning

A logical right shift moves every bit toward the lower-value side. Zeros enter from the left. Bits leaving the right edge are discarded.

The rule

$a \gg k$: move k places right, add k zeros on the left, discard k rightmost bits.

Do it in these small steps

Step 1. Write the width and confirm the value is treated as unsigned.
 Step 2. Move every bit right by k positions.
 Step 3. Insert zeros on the left.
 Step 4. Mark the rightmost bits that were discarded.

Four fully worked examples

Example	Start	Every visible step	Answer
1	4-bit 1100 $\gg$ 1	1100 $\rightarrow$ 0110	0110
2	8-bit 10110000 $\gg$ 3	10110000 $\rightarrow$ 00010110	00010110
3	8-bit 00000111 $\gg$ 2	The two rightmost 1-bits are discarded.	00000001
4	8-bit 0xE4 $\gg$ 4	E4=11100100 $\rightarrow$ 00001110	0x0E

Common mistake

Signed right shift may copy the sign bit instead of inserting zeros. This book uses logical right shift on unsigned rows unless explicitly stated.

Stop and check

The number of inserted zeros on the left must equal the shift count.

3.7 Rotation: move bits around a circle

On this page	After this page
The difference between shifting and rotating.	Rotate a fixed-width word left or right without losing bits.

Plain-language meaning

Rotation moves bits like seats on a circular carousel. A bit leaving one edge immediately re-enters at the other edge. No bit is discarded and no new zero is created.

The rule

Rotate-left by k : take the k leftmost bits and attach them to the right. Rotate-right does the opposite.

Do it in these small steps

Step 1. Write the width and draw a divider after the bits that will wrap.
 Step 2. Move the main block in the requested direction.
 Step 3. Carry the wrapped block to the opposite edge.
 Step 4. Count the number of 1-bits before and after; it must be unchanged.

Four fully worked examples

Example	Start	Every visible step	Answer
1	4-bit rotate-left 1001 by 1	Split 1 001 -> 001 1	0011
2	8-bit rotate-left 10110000 by 3	Split 101 10000 -> 10000 101	10000101
3	8-bit rotate-right 00101101 by 2	Split 001011 01 -> 01 001011	01001011
4	8-bit rotate-left 0xE1 by 4	E1=11100001; swap the two 4-bit halves.	00011110 = 0x1E

Common mistake

A shift loses edge bits; a rotation preserves them. If your rotation changes the count of 1-bits, it is wrong.

Stop and check

Rotate left by k and then right by k . You must recover the original word.

3.8 Exercises: bit operations

How to use this exercise page

Write the truth table beside you. Keep every operand at the stated width. For hex inputs, convert to binary first, perform the operation, then optionally convert back.

Level A - one operation

- T1. 8-bit NOT 00101100.
- T2. 10110110 AND 11110000.
- T3. 00100101 OR 10001000.
- T4. 11100011 XOR 01010101.

Level B - movement

- T5. 8-bit 00110101 << 2.
- T6. 8-bit 11001010 >> 3.
- T7. Rotate-left 8-bit 10100110 by 3.
- T8. Rotate-right 8-bit 00011101 by 2.

Level C - hex and masks

- T9. 0xD3 AND 0x0F. Give binary and hex.
- T10. 0x48 OR 0x83. Give binary and hex.
- T11. 0xA5 XOR 0xFF.
- T12. Rotate-left 0x6B by 4 bits.

Level D - miniature cipher step

- T13. P=01101001, K=11000110. Compute C=P XOR K, then recover P from C.
- T14. Start A=00110111. Compute ((A XOR 10100001) rotate-left 3).
- T15. Start A=0x5C. Compute (A OR 0x03) AND 0xF7.
- T16. Explain in one sentence why rotation is used instead of a shift in many mixing functions.

4 Modular arithmetic: controlled wrap-around

The clock picture

Modulo q means only the position on a q -position clock matters. Numbers separated by a whole number of turns represent the same position. Example modulo 7: ..., -1, 6, 13, 20, ... all point to the same residue 6.

Notation

$a \bmod q$ means the chosen remainder in $0, 1, \dots, q-1$. The statement $a \equiv b \pmod{q}$ means a and b differ by a multiple of q .

4.1 Reduce a number modulo q

On this page	After this page
Finding the canonical remainder, including negative inputs.	Reduce positive and negative integers to 0 through $q-1$.

Plain-language meaning

Reduction means removing complete groups of q . For a positive number, divide by q and keep the remainder. For a negative number, add q repeatedly until the result lies between 0 and $q-1$.

The rule

Choose r with $0 \leq r < q$ and $a-r$ divisible by q .

Do it in these small steps

Step 1. Identify q , the wrap-around size.
 Step 2. Remove or add whole multiples of q .
 Step 3. Stop only when the result is in the interval $0, \dots, q-1$.
 Step 4. Check that the difference from the original is a multiple of q .

Four fully worked examples

Example	Start	Every visible step	Answer
1	$25 \bmod 7$	$25 = 3 \times 7 + 4$	4
2	$83 \bmod 13$	$83 = 6 \times 13 + 5$	5
3	$-3 \bmod 7$	$-3 + 7 = 4$	4
4	$-29 \bmod 11$	$-29 + 33 = 4$, and $33 = 3 \times 11$	4

Common mistake

A negative remainder such as -3 may be useful in some contexts, but when this book asks for the canonical residue, move it into $0, \dots, q-1$.

Stop and check

Is the answer in range? Is original minus answer divisible by q ? Both checks must pass.

4.2 Add and subtract modulo q

On this page	After this page
Ordinary addition/subtraction followed by reduction.	Perform modular addition and subtraction without losing signs.

Plain-language meaning

First calculate as usual. Then reduce the result. You may also reduce inputs first, but do not reduce each digit separately.

The rule

$(a+b) \bmod q$ and $(a-b) \bmod q$: calculate inside the parentheses, then wrap into $0, \dots, q-1$.

Do it in these small steps

Step 1. Write the ordinary integer calculation.
 Step 2. Compute it completely, including any negative sign.
 Step 3. Reduce the single result modulo q .
 Step 4. Check by adding or subtracting one whole q -turn.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$(5+6) \bmod 7$	$5+6=11$; $11-7=4$	4
2	$(12+15) \bmod 13$	27 ; $27-26=1$	1
3	$(3-5) \bmod 7$	$3-5=-2$; $-2+7=5$	5
4	$(19-44) \bmod 11$	$19-44=-25$; $-25+33=8$	8

Common mistake

Do not stop at -2 when the requested representation is 0 through $q-1$.

Stop and check

For subtraction, add q to your result and reduce again. It should represent the same residue.

4.3 Multiply modulo q

On this page	After this page
Product first, reduction second.	Multiply residues and keep the result inside the modulus.

Plain-language meaning

The product can become much larger than q . Multiply normally, then remove whole multiples of q . For large numbers, reducing inputs first can make the product smaller.

The rule

$$(a \times b) \bmod q = ((a \bmod q) \times (b \bmod q)) \bmod q.$$

Do it in these small steps

- Step 1. Optionally reduce both inputs.
- Step 2. Multiply the reduced values.
- Step 3. Reduce the product.
- Step 4. Check that replacing either input by $\text{input}+q$ does not change the answer.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$4 \times 6 \bmod 7$	$4 \times 6 = 24$; $24 - 21 = 3$	3
2	$9 \times 11 \bmod 13$	99 ; $99 - 91 = 8$	8
3	$25 \times 18 \bmod 11$	$25 \bmod 11 = 3$; $18 \bmod 11 = 7$; $3 \times 7 = 21$; $21 \bmod 11 = 10$	10
4	$(-4) \times 7 \bmod 13$	-28 ; add 39 $\rightarrow 11$	11

Common mistake

Reducing means taking the remainder of the whole product, not multiplying the individual decimal digits.

Stop and check

Compute the product once using reduced inputs and once using original inputs. The residues must match.

4.4 Compute powers modulo q

On this page	After this page
Repeated squaring instead of enormous ordinary powers.	Evaluate $a^e \bmod q$ using small intermediate numbers.

Plain-language meaning

Repeated squaring reuses powers $1, 2, 4, 8, \dots$. Square and reduce after every step. Then multiply only the powers whose exponents add to e .

The rule

Build $a^1, a^2, a^4, a^8, \dots$ modulo q . Express e as a sum of powers of 2.

Do it in these small steps

Step 1. Write e as a sum of $1, 2, 4, 8, \dots$.
 Step 2. Compute a^2 by squaring a and reducing.
 Step 3. Square again for a^4 , then a^8 , reducing each time.
 Step 4. Multiply the needed stored powers and reduce.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$3^4 \bmod 7$	$3^2=9 \bmod 7=2$; $3^4=2^2=4$	4
2	$5^5 \bmod 11$	$5^2=25 \bmod 11=3$; $5^4=9$; $5^5=9 \times 5=45 \bmod 11=1$	1
3	$7^{10} \bmod 13$	$7^2=49 \rightarrow 10$; $7^4=100 \rightarrow 9$; $7^8=81 \rightarrow 3$; $10=8+2$: $3 \times 10=30 \rightarrow 4$	4
4	$2^{20} \bmod 17$	$2^4=16 \equiv -1 \bmod 17$; $2^8=1$; $2^{16}=1$; $20=16+4$: $1 \times 16=16$	16

Common mistake

Do not calculate a huge power first on a calculator and only then reduce. Real implementations reduce continuously to keep values bounded.

Stop and check

Write which powers of two add to the exponent. If that line is missing, it is easy to multiply the wrong stored powers.

4.5 Find a modular inverse by search

On this page	After this page
What "division modulo q " really means.	Find a small inverse by testing products.

Plain-language meaning

The inverse of a modulo q is a number b such that $a \times b$ leaves remainder 1. It exists only when a and q have greatest common divisor 1.

The rule

Test $b=1,2,\dots,q-1$ until $(a \times b) \bmod q = 1$.

Do it in these small steps

Step 1. Confirm $\gcd(a,q)=1$ for the examples in this method.

Step 2. Make a small product table $a \times 1, a \times 2, \dots$.

Step 3. Reduce every product modulo q .

Step 4. Stop at the first remainder 1.

Four fully worked examples

Example	Start	Every visible step	Answer
1	Inverse of 3 mod 7	$3 \times 1 = 3$; $3 \times 2 = 6$; $3 \times 3 = 9 \rightarrow 2$; $3 \times 4 = 12 \rightarrow 5$; $3 \times 5 = 15 \rightarrow 1$	5
2	Inverse of 5 mod 11	$5 \times 1 = 5$; $x_2 = 10$; $x_3 = 15 \rightarrow 4$; $x_4 = 20 \rightarrow 9$; $x_5 = 25 \rightarrow 3$; $x_6 = 30 \rightarrow 8$; $x_7 = 35 \rightarrow 2$; $x_8 = 40 \rightarrow 7$; $x_9 = 45 \rightarrow 1$	9
3	Inverse of 7 mod 13	$7 \times 2 = 14 \rightarrow 1$	2
4	Inverse of 8 mod 17	$8 \times 1 = 8$; $x_2 = 16$; $x_3 = 24 \rightarrow 7$; ...; $x_{15} = 120 \rightarrow 1$	15

Common mistake

Not every number has an inverse. Example: 2 has no inverse modulo 6 because every product $2b$ is even and can never be congruent to 1.

Stop and check

Multiply a by your proposed inverse and reduce. The answer must be exactly 1.

4.6 Find an inverse with the extended Euclidean algorithm

On this page	After this page
A systematic method for larger moduli.	Turn gcd calculations into an explicit modular inverse.

Plain-language meaning

Euclid repeatedly divides and keeps remainders until reaching 1. Then we substitute backward to express 1 as a combination of a and q . The coefficient of a is the inverse modulo q .

The rule

If $1 = s*a + t*q$, then $s*a \equiv 1 \pmod{q}$, so $s \pmod{q}$ is the inverse.

Do it in these small steps

Step 1. Run divisions $q = k*a + r$ until the remainder is 1.
 Step 2. Write the final 1 equation.
 Step 3. Substitute earlier remainder equations backward.
 Step 4. Read the coefficient of a and reduce it modulo q .

Four fully worked examples

Example	Start	Every visible step	Answer
1	$3^{-1} \pmod{7}$	$7 = 2 \times 3 + 1 \rightarrow 1 = 7 - 2 \times 3$ Coefficient of 3 is -2; $-2 \pmod{7} = 5$	5
2	$5^{-1} \pmod{11}$	$11 = 2 \times 5 + 1 \rightarrow 1 = 11 - 2 \times 5$ Coefficient -2 $\rightarrow 9 \pmod{11}$	9
3	$17^{-1} \pmod{43}$	$43 = 2 \times 17 + 9$; $17 = 1 \times 9 + 8$; $9 = 1 \times 8 + 1$ $1 = 9 - 8 = 2 \times 9 - 17 = 2 \times 43 - 5 \times 17$	38
4	$37^{-1} \pmod{101}$	$101 = 2 \times 37 + 27$; $37 = 1 \times 27 + 10$; $27 = 2 \times 10 + 7$; $10 = 1 \times 7 + 3$; $7 = 2 \times 3 + 1$ Back-substitution gives $1 = 11 \times 37 - 4 \times 101$	11

Common mistake

The coefficient may be negative. That is not an error; reduce it modulo q at the end.

Stop and check

Always verify by multiplication. For $17 \pmod{43}$: $17 \times 38 = 646$, and $646 \pmod{43} = 1$.

4.7 Exercises: modular arithmetic

How to use this exercise page

Use the two checks repeatedly: the result must be in range, and the difference from the unreduced value must be divisible by q .

Level A - reduction

- M1. $38 \bmod 7$.
- M2. $144 \bmod 13$.
- M3. $-5 \bmod 11$.
- M4. $-47 \bmod 9$.

Level B - operations

- M5. $(9+12) \bmod 8$.
- M6. $(4-17) \bmod 11$.
- M7. $14 \times 19 \bmod 13$.
- M8. $(-6) \times 8 \bmod 17$.

Level C - powers and inverses

- M9. $3^8 \bmod 7$.
- M10. $6^{13} \bmod 17$ using repeated squaring.
- M11. Find $4^{-1} \bmod 9$ by search.
- M12. Find $12^{-1} \bmod 29$ by Euclid.

Level D - chained

- M13. Compute $((17+26) \times 9) \bmod 11$.
- M14. Compute $(5^7 - 3^4) \bmod 13$.
- M15. Solve $7x \equiv 5 \pmod{13}$ using the inverse of 7.
- M16. Explain why 6 has no inverse modulo 15.

5 Polynomials as coefficient arrays

Forget the school-algebra anxiety

In cryptographic code, a polynomial is usually stored as a row of coefficients. The row $[3,2,5]$ means $3 + 2x + 5x^2$. Most operations are array operations with a special rule for multiplying positions.

Index rule

Array index 0 is the constant coefficient, index 1 belongs to x , index 2 belongs to x^2 , and so on.

5.1 Translate between polynomial notation and arrays

On this page	After this page
Reading coefficients in ascending powers of x .	Move safely between formulas and stored arrays.

Plain-language meaning

A coefficient array lists numbers from the constant term upward. Missing powers must be represented by zero so that later indexes do not shift.

The rule

$[a_0, a_1, a_2, \dots]$ means $a_0 + a_1 \cdot x + a_2 \cdot x^2 + \dots$

Do it in these small steps

Step 1. Find the constant term and place it at index 0.
 Step 2. Find the x coefficient and place it at index 1.
 Step 3. Insert zero for every missing power.
 Step 4. Trim or keep trailing zeros according to the stated fixed length.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$3 + 2x + 5x^2$	constant 3, x coefficient 2, x^2 coefficient 5	$[3, 2, 5]$
2	$1 - 4x^2$	x term is missing, so insert 0	$[1, 0, -4]$
3	$7x + x^3$	constant missing $\rightarrow 0$; x^2 missing $\rightarrow 0$	$[0, 7, 0, 1]$
4	$[6, 0, 3, 2]$	index0=6; index1=0; index2=3; index3=2	$6 + 3x^2 + 2x^3$

Common mistake

Do not omit an internal zero. $[1, -4]$ means $1 - 4x$, not $1 - 4x^2$.

Stop and check

Point to index 2. It must always belong to x^2 , even when its coefficient is zero.

5.2 Add and subtract polynomials

On this page	After this page
Position-by-position coefficient operations.	Add or subtract coefficient arrays and optionally reduce modulo q .

Plain-language meaning

Only coefficients attached to the same power of x combine. In arrays, that simply means operate on equal indexes.

The rule

$[a_0, a_1, \dots] \pm [b_0, b_1, \dots] = [a_0 \pm b_0, a_1 \pm b_1, \dots]$.

Do it in these small steps

Step 1. Pad the shorter array with trailing zeros if necessary.
 Step 2. Align index 0 under index 0, index 1 under index 1, etc.
 Step 3. Add or subtract each column.
 Step 4. If a modulus is stated, reduce every resulting coefficient.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$[1, 2] + [3, 4]$	$[1+3, 2+4]$	$[4, 6]$
2	$[5, 0, 2] - [1, 3, 4]$	$[5-1, 0-3, 2-4]$	$[4, -3, -2]$
3	$[1, 2, 3] + [4, 5]$	Pad second: $[4, 5, 0]$ add columns	$[5, 7, 3]$
4	mod 7: $[6, 5, 1] + [4, 4, 6]$	Raw $[10, 9, 7] \rightarrow$ reduce each mod 7	$[3, 2, 0]$

Common mistake

Do not combine different powers, such as adding the x coefficient to the x^2 coefficient.

Stop and check

Convert the arrays back to polynomial notation and check that like powers were combined.

5.3 Multiply a polynomial by a scalar

On this page	After this page
One number applied to every coefficient.	Scale a polynomial and reduce coefficients when needed.

Plain-language meaning

A scalar is one ordinary number. Multiplying by it means multiplying every coefficient by that same number.

The rule

$$c*[a_0, a_1, \dots] = [c*a_0, c*a_1, \dots].$$

Do it in these small steps

Step 1. Copy the coefficient array.
 Step 2. Multiply each coefficient by c.
 Step 3. Keep signs visible.
 Step 4. Reduce each coefficient if a modulus is stated.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$3*[1, 2, 4]$	$[3 \times 1, 3 \times 2, 3 \times 4]$	$[3, 6, 12]$
2	$-2*[3, -1, 5]$	$[-6, 2, -10]$	$[-6, 2, -10]$
3	mod7: $4*[2, 5, 6]$	Raw $[8, 20, 24] \rightarrow [1, 6, 3]$	$[1, 6, 3]$
4	mod11: $9*[7, 0, 8, 3]$	$[63, 0, 72, 27] \rightarrow [8, 0, 6, 5]$	$[8, 0, 6, 5]$

Common mistake

Do not multiply only the first coefficient. A scalar acts on the entire array.

Stop and check

The result must have the same number of coefficient positions as the input.

5.4 Multiply polynomials by convolution

On this page	After this page
Why coefficient indexes add during multiplication.	Compute a small polynomial product as an array.

Plain-language meaning

When x^i is multiplied by x^j , the powers add: x^{i+j} . Therefore coefficient $a[i]$ times $b[j]$ contributes to output position $i+j$. Convolution means adding all contributions that land in the same output position.

The rule

$c[k] = \text{sum of } a[i]*b[j] \text{ over all } i+j=k$. Output length = $\text{len}(a)+\text{len}(b)-1$.

Do it in these small steps

Step 1. Create an output row of zeros with the required length.
 Step 2. Multiply every coefficient of A by every coefficient of B.
 Step 3. Place product $a[i]b[j]$ into output index $i+j$.
 Step 4. Add products that land in the same position.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$[1,1]*[1,1]$	$c_0=1; c_1=1+1=2; c_2=1$	$[1,2,1]$
2	$[2,3]*[4,5]$	$c_0=8; c_1=2*5+3*4=22; c_2=15$	$[8,22,15]$
3	$[1,0,2]*[3,1]$	$c_0=3; c_1=1; c_2=6; c_3=2$	$[3,1,6,2]$
4	mod7: $[3,4,2]*[5,1]$	Raw $[15,23,14,2] \rightarrow \text{reduce mod7}$	$[1,2,0,2]$

Common mistake

Do not multiply only matching indexes. Polynomial multiplication uses every pair of coefficients.

Stop and check

Check the output length before calculating. A length-3 times length-2 product must have length 4 before ring reduction.

5.5 Reduce in the ring $x^n + 1$

On this page	After this page
Wrapping high powers back with a sign change.	Reduce a convolution result to n coefficients.

Plain-language meaning

In the ring modulo x^n+1 , the equation $x^n = -1$ is enforced. Therefore $x^{(n+r)}$ becomes $-x^r$. Coefficients above index $n-1$ wrap back n positions and change sign.

The rule

For every $k \geq n$: coefficient at k is subtracted from position $k-n$. Then discard position k .

Do it in these small steps

Step 1. Start with the full convolution result.
 Step 2. Find every coefficient at index n or higher.
 Step 3. Move it to index $k-n$ and change its sign.
 Step 4. Combine with the coefficient already there; then reduce modulo q if stated.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$n=3$: [1,2,3,4]	$4x^3 \rightarrow -4$; constant becomes 1-4	[-3,2,3]
2	$n=4$: [2,0,1,3,5,6]	$5x^4 \rightarrow -5$ at index0; $6x^5 \rightarrow -6$ at index1	[-3,-6,1,3]
3	$n=3$: [0,1,2,3,4]	$3x^3 \rightarrow -3$ index0; $4x^4 \rightarrow -4$ index1	[-3,-3,2]
4	$\text{mod } 7, n=3$: [5,6,4,3,2]	Wrap: [5-3,6-2,4]=[2,4,4]; already in range	[2,4,4]

Common mistake

The rule $x^n = -1$ belongs specifically to modulus x^n+1 . A ring x^n-1 would wrap without the sign change.

Stop and check

After reduction, exactly n coefficient positions must remain.

5.6 Exercises: polynomial arithmetic

How to use this exercise page

Write arrays with indexes 0,1,2,... above them. For convolution, make an empty output row before multiplying.

Level A - representation

- P1. Write $4+3x^2$ as an array.
- P2. Write $[2,-1,0,5]$ as a polynomial.
- P3. Add $[3,1,4]$ and $[2,5,0]$.
- P4. Subtract $[1,4,2]$ from $[6,2,9]$.

Level B - scaling and modulus

- P5. Compute $5*[1,-2,3]$.
- P6. Mod 7, compute $3*[6,4,2]$.
- P7. Mod 11, add $[9,8,7]$ and $[6,5,4]$.
- P8. Mod 13, subtract $[8,9,10]$ from $[2,3,4]$.

Level C - convolution

- P9. $[1,2]*[3,4]$.
- P10. $[2,0,1]*[1,3]$.
- P11. Mod 7, $[5,2]*[4,6]$.
- P12. $[1,1,1]*[1,1,1]$.

Level D - ring reduction

- P13. Reduce $[2,3,4,5]$ in x^3+1 .
- P14. Reduce $[1,0,2,3,4,5]$ in x^4+1 .
- P15. Multiply $[1,2,1]$ by $[2,1]$ and reduce in x^3+1 .
- P16. Mod 7, multiply $[3,5,1]$ by $[4,2]$ and reduce in x^3+1 .

6 Vectors and matrices: repeated dot products

The essential idea

A dot product multiplies matching positions and adds. A matrix-vector product is simply one dot product per matrix row. FrodoKEM uses very large versions of this pattern; lattice schemes use similar products whose entries are polynomials.

6.1 Compute a dot product

On this page	After this page
Multiply matching entries, then add.	Evaluate a dot product with signs and a modulus.

Plain-language meaning

Two equal-length vectors are aligned. Multiply each vertical pair. Add all pair-products to get one number.

The rule

$[a_0, a_1, \dots] \text{ dot } [b_0, b_1, \dots] = a_0 \cdot b_0 + a_1 \cdot b_1 + \dots$

Do it in these small steps

Step 1. Confirm equal lengths.
 Step 2. Multiply entries at equal indexes.
 Step 3. Write every product with its sign.
 Step 4. Add; then reduce modulo q if stated.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$[1, 2] \text{ dot } [3, 4]$	$1 \times 3 + 2 \times 4 = 3 + 8$	11
2	$[2, -1, 3] \text{ dot } [4, 5, 2]$	$8 - 5 + 6$	9
3	$[0, 7, 2, 1] \text{ dot } [9, 3, 4, 5]$	$0 + 21 + 8 + 5$	34
4	mod11: $[5, 8, 3] \text{ dot } [7, 4, 9]$	$35 + 32 + 27 = 94; 94 \bmod 11 = 6$	6

Common mistake

Do not multiply every entry of the first vector by every entry of the second. That would be a different operation.

Stop and check

The dot product of two length- n vectors is one number, not another length- n vector.

6.2 Multiply a matrix by a vector

On this page	After this page
One dot product per row.	Calculate a matrix-vector result without losing row order.

Plain-language meaning

A matrix is a stack of rows. Dot each row with the vector. The first row creates the first output entry, the second row creates the second output entry, and so on.

The rule

If A has m rows and n columns and v has n entries, then $A*v$ has m entries.

Do it in these small steps

Step 1. Check that each row length equals the vector length.
 Step 2. Dot row 1 with the vector.
 Step 3. Dot row 2 with the vector, continuing for every row.
 Step 4. Place the row results into one output column/vector in the same order.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$[[1,2],[3,4]] * [5,6]$	row1: $5+12=17$ row2: $15+24=39$	$[17,39]$
2	$[[2,0,-1],[1,3,2]] * [4,5,6]$	row1: $8+0-6=2$ row2: $4+15+12=31$	$[2,31]$
3	$[[1,1],[1,-1],[2,3]] * [7,2]$	9; 5; $14+6=20$	$[9,5,20]$
4	$[[3,2,1],[0,4,5]] * [2,1,3]$	$6+2+3=11$ $0+4+15=19$	$[11,19]$

Common mistake

Do not mix entries from different rows. Finish one row dot product before moving to the next.

Stop and check

The output length must equal the number of matrix rows.

6.3 Matrix-vector multiplication modulo q

On this page	After this page
Reducing each row result.	Compute the same matrix pattern used in toy lattice examples.

Plain-language meaning

The dot products are ordinary integer calculations, followed by modular reduction. You may reduce intermediate sums when convenient, but the final residue must be the same.

The rule

For each row i : $y[i] = (\sum_j A[i,j]*v[j]) \bmod q$.

Do it in these small steps

Step 1. Compute one row dot product.
 Step 2. Keep the unreduced integer visible.
 Step 3. Reduce that row result modulo q .
 Step 4. Repeat independently for each row.

Four fully worked examples

Example	Start	Every visible step	Answer
1	mod7: $[[1,2],[3,4]]*[5,6]$	raw $[17,39] \rightarrow [3,4]$	$[3,4]$
2	mod11: $[[2,5,1],[7,3,4]]*[6,2,8]$	row1 = $12+10+8=30 \rightarrow 8$ row2 = $42+6+32=80 \rightarrow 3$	$[8,3]$
3	mod13: $[[4,0,9],[1,8,2]]*[3,7,5]$	$57 \rightarrow 5$; $3+56+10=69 \rightarrow 4$	$[5,4]$
4	mod17: $[[9,9],[16,3],[4,12]]*[5,7]$	$108 \rightarrow 6$; $101 \rightarrow 16$; $104 \rightarrow 2$	$[6,16,2]$

Common mistake

Do not reduce the matrix indexes or the vector length. Only numerical entries and sums are reduced.

Stop and check

Recompute one row by reducing each product first. The final residue should match.

6.4 Exercises: vectors and matrices

How to use this exercise page

Draw a horizontal line under each row after finishing its dot product. This prevents row results from bleeding into one another.

Level A - dot products

- V1. $[2,3]$ dot $[4,5]$.
- V2. $[1,-2,4]$ dot $[3,5,2]$.
- V3. $[0,6,1,2]$ dot $[9,2,7,3]$.
- V4. Mod 7: $[5,4,6]$ dot $[3,2,1]$.

Level B - matrices

- V5. $[[1,3],[2,4]]*[5,2]$.
- V6. $[[2,0,1],[3,1,-1]]*[4,5,6]$.
- V7. $[[1,1],[2,0],[0,3]]*[7,4]$.
- V8. $[[4,2,1],[1,5,3]]*[2,3,1]$.

Level C - modular

- V9. Mod 7: $[[2,3],[4,5]]*[6,1]$.
- V10. Mod 11: $[[7,1,4],[2,8,5]]*[3,6,2]$.
- V11. Mod 13: $[[5,9],[12,4],[3,7]]*[8,6]$.
- V12. A lattice toy step uses $b=A*s+e \bmod 7$ with $A=[[1,2],[4,3]]$, $s=[2,1]$, $e=[1,-1]$. Compute b .

7 Centering, rounding, compression, and small noise

Why approximate values appear

Several post-quantum schemes deliberately round or compress coefficients. The receiver does not recover every internal number exactly; it recovers the intended message because the added noise and rounding error stay inside a safe margin.

7.1 Choose a centered representative

On this page	After this page
Writing residues near zero instead of only 0 through $q-1$.	Convert canonical residues into signed values around zero.

Plain-language meaning

Modulo q , the residue $q-1$ can also be written as -1 , $q-2$ as -2 , and so on. A centered representation chooses the equivalent number closest to zero.

The rule

For odd q , use the interval $-(q-1)/2$ through $+(q-1)/2$. If $r > q/2$, replace r by $r-q$.

Do it in these small steps

Step 1. Reduce to $0, \dots, q-1$ first.
 Step 2. Compare r with $q/2$.
 Step 3. If r is in the upper half, subtract q .
 Step 4. Verify that the centered value is congruent to the original.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$6 \bmod 7$	$6 > 3.5$, so $6-7$	-1
2	$5 \bmod 11$	5 is already near zero	5
3	$9 \bmod 11$	$9 > 5.5$, so $9-11$	-2
4	$15 \bmod 17$	$15 > 8.5$, so $15-17$	-2

Common mistake

Centered representatives are still residues. -2 and 9 are the same modulo 11 ; the notation changes, not the residue class.

Stop and check

Add q to any negative centered value and confirm that you recover its canonical residue.

7.2 Round to the nearest integer

On this page	After this page
Turning a real ratio into a discrete value.	Perform explicit nearest-integer rounding and identify ties.

Plain-language meaning

Rounding chooses the nearest whole number. Values below .5 go down; values above .5 go up. Exact .5 ties require a stated convention. In toy examples here, ties round upward unless noted.

The rule

$\text{round}(n/d)$: compute the decimal or compare remainder with $d/2$.

Do it in these small steps

Step 1. Divide or write $n = k \cdot d + r$.
 Step 2. The unrounded value is $k + r/d$.
 Step 3. Compare r/d with $1/2$.
 Step 4. Choose k or $k+1$ according to the stated tie rule.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$\text{round}(17/5)$	$17 = 3 \times 5 + 2$; $2/5 = 0.4$	3
2	$\text{round}(19/5)$	$19 = 3 \times 5 + 4$; $4/5 = 0.8$	4
3	$\text{round}(13/4)$	$13 = 3 \times 4 + 1$; $1/4 = 0.25$	3
4	$\text{round}(15/4)$	$15 = 3 \times 4 + 3$; $3/4 = 0.75$	4

Common mistake

Do not silently change the tie rule. Cryptographic specifications define exact integer formulas so every implementation agrees.

Stop and check

Write the fractional distance to both neighboring integers. The selected one must be no farther away.

7.3 Compress a residue to d bits

On this page	After this page
Mapping q possible values into 2^d buckets.	Calculate a toy compression value.

Plain-language meaning

Compression scales a residue from the range $0, \dots, q-1$ into the smaller range $0, \dots, 2^d-1$. Multiple original values may land in the same bucket.

The rule

Toy formula: $\text{Compress}_{q,d}(x) = \text{round}((2^d / q) * x) \bmod 2^d$.

Do it in these small steps

Step 1. Reduce x modulo q .
 Step 2. Compute 2^d .
 Step 3. Multiply x by 2^d and divide by q .
 Step 4. Round, then reduce modulo 2^d .

Four fully worked examples

Example	Start	Every visible step	Answer
1	$q=17, d=3, x=0$	$8 \times 0 / 17 = 0$	0
2	$q=17, d=3, x=4$	$32 / 17 = 1.882 \rightarrow \text{round } 2$	2
3	$q=17, d=3, x=9$	$72 / 17 = 4.235 \rightarrow 4$	4
4	$q=17, d=3, x=16$	$128 / 17 = 7.529 \rightarrow 8; 8 \bmod 8 = 0$	0

Common mistake

The top residue can wrap to bucket 0 because the modular range is circular. This is expected in formulas that include $\bmod 2^d$.

Stop and check

The compressed answer must fit in d bits. For $d=3$ it must lie in $0, \dots, 7$.

7.4 Decompress a d-bit value

On this page	After this page
Scaling a bucket back into the q-range.	Compute the representative recovered from compressed data.

Plain-language meaning

Decompression chooses a q-range representative near the center of the selected bucket. It usually does not recover the exact original x.

The rule

Toy formula: $\text{Decompress}_{q,d}(y) = \text{round}((q / 2^d) * y)$.

Do it in these small steps

Step 1. Confirm y lies in $0, \dots, 2^d - 1$.
 Step 2. Multiply y by q.
 Step 3. Divide by 2^d .
 Step 4. Round according to the defined integer rule.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$q=17, d=3, y=0$	$17 \times 0 / 8 = 0$	0
2	$q=17, d=3, y=2$	$34 / 8 = 4.25 \rightarrow 4$	4
3	$q=17, d=3, y=4$	$68 / 8 = 8.5 \rightarrow 9$ with upward ties	9
4	$q=17, d=3, y=7$	$119 / 8 = 14.875 \rightarrow 15$	15

Common mistake

Decompression is not an inverse in the ordinary one-to-one sense. Information was deliberately discarded during compression.

Stop and check

Compress your decompressed value again. It should normally return the same bucket y.

7.5 Add small noise and recover a coarse value

On this page	After this page
Why small errors do not necessarily destroy the message.	Track a toy signal through noise and threshold decoding.

Plain-language meaning

A message bit can be represented by two well-separated regions, such as near 0 and near $q/2$. Small noise moves the value slightly, but a threshold can still identify the intended region.

The rule

Toy encoding for $q=16$: bit 0 $\rightarrow$ 0, bit 1 $\rightarrow$ 8. Add small e modulo 16. Decode 1 when the centered value is closer to 8 than to 0.

Do it in these small steps

Step 1. Encode the bit as 0 or 8.
 Step 2. Add the signed noise e .
 Step 3. Reduce modulo 16.
 Step 4. Compare circular distance to 0 and to 8.

Four fully worked examples

Example	Start	Every visible step	Answer
1	bit0, $e=+2$	$0+2=2$; distance to0=2, to8=6	decode 0
2	bit0, $e=-3$	$0-3=-3 \rightarrow 13$; circular distance to0=3, to8=5	decode 0
3	bit1, $e=+2$	$8+2=10$; distance to8=2, to0=6	decode 1
4	bit1, $e=-3$	$8-3=5$; distance to8=3, to0=5	decode 1

Common mistake

This toy threshold is not a real scheme parameter set. Real schemes choose bounds carefully and may fail with very small probability when noise becomes too large.

Stop and check

Ask which two regions represent the messages and what maximum noise still keeps them separated.

7.6 Exercises: rounding and noise

How to use this exercise page

Show the unrounded fraction. A bare final integer is not enough because most mistakes happen in the scaling step.

Level A - centered and rounded

- R1. Center 8 modulo 11.
- R2. Center 12 modulo 17.
- R3. $\text{round}(23/6)$.
- R4. $\text{round}(29/8)$.

Level B - compression $q=17, d=3$

- R5. Compress $x=2$.
- R6. Compress $x=6$.
- R7. Compress $x=11$.
- R8. Compress $x=15$.

Level C - decompression $q=17, d=3$

- R9. Decompress $y=1$.
- R10. Decompress $y=3$.
- R11. Decompress $y=5$.
- R12. Decompress $y=6$.

Level D - signal plus noise $q=16$

- R13. Encode bit0 and add $e=+3$. Decode.
- R14. Encode bit1 and add $e=-2$. Decode.
- R15. Encode bit0 and add $e=+5$. Which region is now closer?
- R16. Explain why compression can be useful even though exact coefficients are lost.

8 States, permutations, sponges, and streams

A state is just organized working memory

When a specification says "initialize the state," imagine a fixed row or grid of numbered boxes. Each round reads those boxes and writes transformed values back. The word sounds abstract, but the data structure is concrete.

8.1 Apply a toy state permutation

On this page	After this page
A reversible fixed-size transformation.	Follow a small round without losing old values.

Plain-language meaning

A permutation changes the entire state but is reversible. Toy round: for state [a,b,c,d], first compute temporary values from the old state, then replace the state. Temporaries matter because overwriting a too early would corrupt later calculations.

The rule

Toy $P([a,b,c,d]) = [b, c, d, (a \text{ XOR } b)]$ using 4-bit words.

Do it in these small steps

- Step 1. Copy the complete old state.
- Step 2. Compute the new positions from the old values only.
- Step 3. Store results in a separate temporary row.
- Step 4. Replace the old state only after every new entry is ready.

Four fully worked examples

Example	Start	Every visible step	Answer
1	[1,2,3,4]	new=[2,3,4,1 XOR 2=3]	[2,3,4,3]
2	[0xA,0x5,0x0,0xF]	A XOR 5 = F	[5,0,F,F]
3	[7,7,1,2]	7 XOR 7=0	[7,1,2,0]
4	[0xC,0x3,0x9,0x6]	C=1100, 3=0011, XOR=1111=F	[3,9,6,F]

Common mistake

If you overwrite a before computing a XOR b, later output may accidentally use the new a. Use temporaries unless the update order is proven safe.

Stop and check

Can you identify which outputs are moves and which output is newly mixed?

8.2 Absorb and squeeze in a toy sponge

On this page	After this page
Rate, capacity, padding, and repeated permutation.	Trace a tiny sponge for several message blocks.

Plain-language meaning

A sponge state is split into a visible input/output part called the rate and a hidden working part called the capacity. To absorb, XOR a message block into the rate and apply the permutation. To squeeze, read from the rate; permute again when more output is needed.

The rule

Toy state $[r0, r1 \mid c0, c1]$, all 4-bit. Absorb block $[m0, m1]$: $[r0 \text{ XOR } m0, r1 \text{ XOR } m1 \mid c0, c1]$, then apply P.

Do it in these small steps

Step 1. Initialize every state cell exactly as specified.
 Step 2. Split the message into rate-sized blocks and pad the final block.
 Step 3. XOR one block into the rate only.
 Step 4. Apply P after each absorbed block; later read rate cells for output.

Four fully worked examples

Example	Start	Every visible step	Answer
1	state $[0,0 \mid 0,0]$, block $[1,2]$	XOR $\rightarrow [1,2 \mid 0,0]$; P toy $\rightarrow [2,0,0,3]$	state $[2,0 \mid 0,3]$
2	then block $[3,4]$	rate XOR $\rightarrow [1,4 \mid 0,3]$; P $\rightarrow [4,0,3,5]$	state $[4,0 \mid 3,5]$
3	squeeze one block	Read rate $[4,0]$. No state change yet.	output $[4,0]$
4	need another block	Apply P to $[4,0,3,5] \rightarrow [0,3,5,4]$; read rate	output $[0,3]$

Common mistake

Do not XOR the message into the capacity unless the construction explicitly says so. The rate and capacity have different roles.

Stop and check

Mark the vertical rate/capacity divider on every written state. This prevents accidental indexing mistakes.

8.3 Encrypt and decrypt with a stream using XOR

On this page	After this page
Keystream generation separated from message data.	Apply the same XOR operation for encryption and decryption.

Plain-language meaning

A stream cipher creates pseudorandom-looking bytes from a key, nonce, and counter. Encryption XORs plaintext with that keystream. Decryption XORs ciphertext with the identical keystream.

The rule

$C = P \text{ XOR } K_{\text{stream}}$ and $P = C \text{ XOR } K_{\text{stream}}$. Never reuse the same keystream for two different plaintexts.

Do it in these small steps

Step 1. Generate or receive a keystream of the same length as the message.
 Step 2. Align plaintext and keystream bytes.
 Step 3. XOR each pair to create ciphertext.
 Step 4. For decryption, regenerate the identical keystream and XOR again.

Four fully worked examples

Example	Start	Every visible step	Answer
1	$P=0x3A$, $K=0xC7$	$00111010 \text{ XOR } 11000111 = 11111101$	$C=0xFD$
2	$C=0xFD$, $K=0xC7$	$11111101 \text{ XOR } 11000111 = 00111010$	$P=0x3A$
3	$P=[0x10, 0x20]$, $K=[0xAA, 0x0F]$	$10 \text{ XOR } AA = BA$; $20 \text{ XOR } 0F = 2F$	$C=[BA, 2F]$
4	decrypt $[BA, 2F]$	$BA \text{ XOR } AA = 10$; $2F \text{ XOR } 0F = 20$	$P=[10, 20]$

Common mistake

If the same keystream K encrypts $P1$ and $P2$, then $C1 \text{ XOR } C2 = P1 \text{ XOR } P2$. This leaks a direct relation between the plaintexts.

Stop and check

Encrypt and immediately decrypt your example. The exact original bytes must return.

8.4 Exercises: states and streams

How to use this exercise page

Write old state and new state on separate lines. Never update in place on paper unless the rule explicitly permits it.

Level A - toy permutation

- S1. Apply $P([a,b,c,d])=[b,c,d,a \text{ XOR } b]$ to $[1,4,2,7]$.
- S2. Apply P to $[0xF,0x3,0x8,0x1]$.
- S3. Apply P twice to $[2,5,6,1]$.
- S4. Which old cells determine the final new cell?

Level B - sponge

- S5. Start $[0,0|0,0]$, absorb $[2,5]$, then apply P .
- S6. Continue S5 by absorbing $[1,7]$.
- S7. From the state after S6, squeeze two rate cells.
- S8. Explain the difference between rate and capacity in one concrete sentence.

Level C - stream XOR

- S9. Encrypt $P=0x56$ with $K=0xB1$.
- S10. Decrypt $C=0xE7$ with $K=0xB1$.
- S11. Encrypt $[0x00,0xFF,0x35]$ with $[0xA5,0x0F,0xC0]$.
- S12. Show algebraically with XOR identities why decryption restores P .

9 Nim preview: from calculations to buffers

Purpose of this preview

Later optimization volumes will read actual Tyr-Crypto code. This preview explains only the syntax and memory ideas needed to recognize calculations. It is not a complete Nim language course.

Code-reading rule

Read types first, loop bounds second, array indexes third, and arithmetic last. This order tells you the width, how many repetitions occur, where data lives, and only then what transformation happens.

9.1 Variables and fixed-width integer types

On this page	After this page
What var, let, uint8, uint32, and suffixes mean.	Read four common Nim declarations from cryptographic code.

Plain-language meaning

A variable is a named storage location. var means the value may change. let means it is assigned once. uint8 stores an unsigned 8-bit value; uint32 stores 32 bits.

The rule

var name: Type = value. A suffix such as 1'u32 forces the literal to be uint32.

Do it in these small steps

Step 1. Locate var or let.
 Step 2. Read the name after it.
 Step 3. Read the type after the colon and translate the bit width.
 Step 4. Read the initial value after = and note whether later assignment is allowed.

Four fully worked examples

Example	Start	Every visible step	Answer
1	var x: uint8 = 5	mutable x; 8 unsigned bits; starts at 5	x may change
2	let q: uint32 = 3329	fixed binding q; 32-bit unsigned	q assigned once
3	var flag = true	type inferred as bool	mutable Boolean
4	let one = 1'u32	suffix fixes literal width/type	uint32 value 1

Common mistake

The type width is part of the calculation. A uint8 value has only 256 possible bit patterns; a uint32 has 2^{32} .

Stop and check

Before reading an operation, can you state the width and whether the variable may be overwritten?

9.2 Fixed arrays and indexing

On this page	After this page
How coefficient rows and states appear in Nim.	Read array sizes and map indexes to positions.

Plain-language meaning

An array has a fixed number of equally typed elements. Nim indexes start at 0. Therefore an array of length 4 has valid indexes 0,1,2,3.

The rule

`array[N,T]` means N elements of type T. `a[i]` selects index i.

Do it in these small steps

Step 1. Read N, the number of cells.
 Step 2. Read T, the type of every cell.
 Step 3. List valid indexes from 0 to N-1.
 Step 4. Trace each assignment to the exact cell.

Four fully worked examples

Example	Start	Every visible step	Answer
1	<code>var a: array[4,uint8]</code>	4 cells, each 8 bits; indexes 0..3	fixed 4-byte array
2	<code>a[0] = 7</code>	write 7 into first cell	[7,0,0,0] initially
3	<code>a[3] = a[0] xor 2</code>	7 XOR2=5; write at last cell	[7,0,0,5]
4	<code>let x = a[1]</code>	read second cell, not first	x receives 0

Common mistake

Index 4 is outside an `array[4,T]`. The length is 4, but the final valid index is 3.

Stop and check

For every array declaration, write the valid index interval beside it.

9.3 Loops with public fixed bounds

On this page	After this page
Reading <code>..<code><</code></code> and repeated array work.	Trace small loops and count exact iterations.

Plain-language meaning

A loop repeats a block. `0 ..< n` includes 0 but excludes n, so it produces n iterations: 0,1,...,n-1. Fixed public bounds are common in constant-time code.

The rule

`for i in 0 ..< n`: repeats the indented body for `i=0` through `n-1`.

Do it in these small steps

Step 1. Write the sequence of `i` values before tracing the body.
 Step 2. Execute the indented lines once for each `i`.
 Step 3. Update only the indexed cell named by the current `i`.
 Step 4. After the final iteration, inspect the complete array.

Four fully worked examples

Example	Start	Every visible step	Answer
1	<code>for i in 0 ..< 4:</code> <code> a[i]=1</code>	<code>i=0,1,2,3</code> ; set every cell	<code>[1,1,1,1]</code>
2	<code>for i in 0 ..< 4:</code> <code> a[i]=uint8(i)</code>	write index as value	<code>[0,1,2,3]</code>
3	<code>for i in 0 ..< 4:</code> <code> a[i]=a[i] xor 1</code>	toggle lowest bit in each entry	<code>[1,0,3,2]</code> from <code>[0,1,2,3]</code>
4	<code>var s=0; for i in 0 ..< 4: s+=i</code>	<code>s=0+0+1+2+3</code>	6

Common mistake

Do not include `n` in a `..<` loop. For `0..<4`, `i=4` never occurs.

Stop and check

Count iterations before tracing. The count should equal `n` for `0..<n`.

9.4 Unsigned wrap-around and explicit rotation

On this page	After this page
How fixed-width arithmetic differs from unlimited integers.	Recognize overflow as modular arithmetic rather than a mysterious error.

Plain-language meaning

An unsigned w -bit word naturally represents residues modulo 2^w . When an operation exceeds the maximum, low-level cryptographic code often intentionally keeps only the lowest w bits.

The rule

For uint8 toy arithmetic: result wraps modulo 256. For uint32: modulo 2^{32} . Rotation is built from shifts and OR using the fixed width.

Do it in these small steps

Step 1. Identify the word width w .
 Step 2. Perform the ordinary integer operation.
 Step 3. Reduce modulo 2^w when wrap-around is intended.
 Step 4. Write the resulting fixed-width bit pattern.

Four fully worked examples

Example	Start	Every visible step	Answer
1	uint8 250+10	260 mod256	4
2	uint8 3-5	-2 mod256	254
3	uint8 20x20	400 mod256	144
4	8-bit rotl 0xB1 by3	(B1<<3 low8) OR (B1>>5) 0x88 OR0x05	0x8D

Common mistake

Nim may require explicit operators or compiler settings for desired wrapping behavior. Never assume overflow semantics without reading the exact type and operator.

Stop and check

Convert the wrapped result to bits and confirm that only the lowest w bits remain.

9.5 Buffers, stack, heap, and secret lifetime

On this page	After this page
Where bytes live and why cleanup is difficult.	Distinguish four common memory situations in cryptographic code.

Plain-language meaning

RAM is a large collection of addressable storage cells. A buffer reserves a consecutive group. Stack storage usually follows function scope and has predictable lifetime; heap storage is managed dynamically and may survive longer. Copies, compiler optimizations, and garbage collection can make secret erasure nontrivial.

The rule

Track: allocation -> writes -> copies -> reads -> last use -> wipe attempt -> end of lifetime. Security depends on the entire path, not only the final wipe line.

Do it in these small steps

Step 1. Identify where the buffer is created.
 Step 2. Mark whether its size is fixed or dynamic.
 Step 3. Mark every copy or returned value that may duplicate data.
 Step 4. Locate last use and cleanup; ask whether the compiler can remove the wipe.

Four fully worked examples

Example	Start	Every visible step	Answer
1	<code>var key: array[32,byte] inside proc</code>	fixed local buffer; typically stack-like storage	short scoped key array
2	<code>var data = newSeq[byte](n)</code>	dynamic sequence; backing storage on heap	heap buffer
3	<code>let copy = key</code>	entire fixed array copied to a second location	two secret copies
4	<code>for i in 0..<code>key.len</code>: volatileStore(key[i],0)</code>	explicit volatile-style overwrite attempt	wipe before scope ends

Common mistake

A wipe of one variable does not erase earlier copies, temporary registers, allocator remnants, swap, crash dumps, or compiler-created temporaries.

Stop and check

Draw a lifetime line for each sensitive buffer. Can you name its creation, final use, and every obvious copy?

9.6 Exercises: reading tiny Nim fragments

How to use this exercise page

Do not run the code first. Predict on paper, then use execution only as a check. The goal is to read calculations rather than experiment blindly.

Level A - declarations and arrays

- N1. What widths and valid indexes are created by `var s: array[8,uint32]`?
- N2. After `var a=[1'u8,2,3,4]; a[2]=9`, what is `a`?
- N3. Explain the difference between `var` and `let`.
- N4. What is the final valid index of `array[16,byte]`?

Level B - loops

- N5. Trace `var a: array[4,uint8]; for i in 0..<4: a[i]=uint8(2*i)`.
- N6. Trace `var s=1; for i in 0..<3: s=s*2`.
- N7. How many iterations does `for i in 0..<256` perform?
- N8. Why can a public fixed loop bound be preferable to stopping when a secret-dependent condition becomes true?

Level C - width and lifetime

- N9. Compute uint8 wrap-around for `240+30`.
- N10. Compute uint8 wrap-around for `1-3`.
- N11. A secret array is copied into two temporaries and only the original is wiped. How many obvious copies remain?
- N12. List four places where secret data may persist despite one source-level wipe.

10 Cumulative guided calculations

How cumulative pages differ

These exercises combine several earlier methods. The scaffold deliberately names the next subtask, so you can focus on one operation at a time. In later algorithm volumes, the same format will grow into full key generation, encapsulation, signing, and verification traces.

10.1 Cumulative A: byte mixing pipeline

How to use this exercise page

Keep every value as an 8-bit row. Complete one numbered line before reading the next.

Pipeline C1

Start with $x=0x3C$ and $k=0xA7$.

C1.1 Convert x and k to binary.

C1.2 Compute $y=x \text{ XOR } k$.

C1.3 Rotate y left by 3 bits.

C1.4 AND the result with $0xF7$.

C1.5 Give the final binary, hex, and decimal values.

Pipeline C2

Start with $x=11001001$.

C2.1 Compute NOT x .

C2.2 OR with 00000101 .

C2.3 Rotate right by 2.

C2.4 XOR with the original x .

C2.5 Convert the result to hex.

10.2 Cumulative B: toy lattice pipeline

How to use this exercise page

This is not real Kyber or SABER. It combines the same kinds of subcalculations: matrix-vector multiplication, noise addition, modular reduction, and compression.

Pipeline C3

Use $q=17$, $d=3$, $A=[[3,5],[7,2]]$, $s=[4,1]$, $e=[-1,2]$.

C3.1 Compute $A*s$ as ordinary integers.

C3.2 Add e component by component.

C3.3 Reduce both components modulo 17 to obtain b .

C3.4 Compress each component to 3 bits.

C3.5 Decompress both buckets. Compare with b using centered differences.

Pipeline C4

Use $q=7$ and ring x^3+1 . Let $a=[2,4,1]$, $s=[3,1,2]$, $e=[1,-1,0]$.

C4.1 Convolve $a*s$.

C4.2 Reduce in x^3+1 .

C4.3 Add e .

C4.4 Reduce every coefficient modulo 7.

C4.5 Write the final array as a polynomial.

10.3 Cumulative C: toy sponge and stream

How to use this exercise page

Use the toy $P([a,b,c,d])=[b,c,d,a \text{ XOR } b]$ on 4-bit cells. Keep the rate/capacity divider visible.

Pipeline C5

Initialize state $[0,0|0,0]$. Absorb blocks $[0xA,0x3]$ and $[0x5,0xC]$.

C5.1 XOR the first block into the rate and apply P.

C5.2 XOR the second block into the new rate and apply P.

C5.3 Squeeze the first two-cell output block.

C5.4 Apply P and squeeze a second block.

C5.5 Use the four squeezed nibbles as two bytes of keystream and encrypt plaintext $[0x42,0x19]$.

11 Compact solutions

How to use solutions

A solution is a diagnostic tool, not a replacement for writing. Compare the first line where your work differs. Later lines are usually consequences of that first mismatch.

11.1 Number-system solutions

Exercises B1-B16

No.	Answer and short path
B1	$101 = 4 + 1 = 5$.
B2	$100000 = 32$.
B3	$01101100 = 64 + 32 + 8 + 4 = 108$.
B4	$11110001 = 128 + 64 + 32 + 16 + 1 = 241$.
B5	$9 = 8 + 1 \rightarrow 1001$.
B6	$26 = 16 + 8 + 2 \rightarrow 11010$.
B7	$73 = 64 + 8 + 1 \rightarrow 01001001$.
B8	$255 \rightarrow 11111111$.
B9	$5 \rightarrow 0101$, $D \rightarrow 1101$: 01011101 .
B10	$C03 \rightarrow 1100\ 0000\ 0011$.
B11	$0111\ 1110 \rightarrow 0x7E$.
B12	$\text{Pad } 0001\ 0110\ 1001 \rightarrow 0x169$.
B13	$0x7B = 7 \times 16 + 11 = 123$.
B14	$210 = 11010010 = 0xD2$.
B15	$0x123 = 0001\ 0010\ 0011$ and $1 \times 256 + 2 \times 16 + 3 = 291$.
B16	$11101010 = 234 = 0xEA$.

11.2 Bit-operation solutions

Exercises T1-T16

No.	Answer and short path
T1	NOT 00101100 = 11010011.
T2	10110110 AND 11110000 = 10110000.
T3	00100101 OR 10001000 = 10101101.
T4	11100011 XOR 01010101 = 10110110.
T5	00110101 <<2 = 11010100.
T6	11001010 >>3 = 00011001.
T7	Split 101 00110 -> 00110101.
T8	Split 000111 01 -> 01000111.
T9	D3=11010011; 0F=00001111; AND=00000011=0x03.
T10	48 OR83: 01001000 OR10000011=11001011=0xCB.
T11	A5 XOR FF = 5A.
T12	Rotate by 4 swaps nibbles: 0xB6.
T13	C=01101001 XOR11000110=10101111. C XOR K returns 01101001.
T14	XOR:10010110; rotl3:10110100.
T15	5C OR03=5F; 5F ANDF7=57.
T16	Rotation preserves all bits while spreading their positions; shifting discards bits and inserts zeros.

11.3 Modular-arithmetic solutions

Exercises M1-M16

No.	Answer and short path
M1	$38 = 5 \times 7 + 3 \rightarrow 3$.
M2	$144 = 11 \times 13 + 1 \rightarrow 1$.
M3	$-5 + 11 = 6$.
M4	$-47 + 54 = 7$.
M5	$21 \bmod 8 = 5$.
M6	$-13 \bmod 11 = 9$.
M7	$266 \bmod 13 = 6$.
M8	$-48 \bmod 17 = 3$.
M9	$3^2 = 2 \bmod 7$; $3^4 = 4$; $3^8 = 2$.
M10	$6^2 = 2$; $6^4 = 4$; $6^8 = 16$; $13 = 8 + 4 + 1$; $16 \times 4 \times 6 = 384 \bmod 17 = 10$.
M11	$4 \times 7 = 28 \equiv 1 \bmod 9$, so inverse 7.
M12	$29 = 2 \times 12 + 5$; $12 = 2 \times 5 + 2$; $5 = 2 \times 2 + 1$. Back: $1 = 5 - 2(12 - 2 \times 5) = 5 \times 5 - 2 \times 12 = 5(29 - 2 \times 12) - 2 \times 12 = 5 \times 29 - 12 \times 12$. Inverse $-12 \equiv 17$.
M13	$17 + 26 = 43$; $43 \times 9 = 387$; $387 \bmod 11 = 2$.
M14	$5^7 \bmod 13 = 8$; $3^4 \bmod 13 = 3$; difference 5.
M15	$7^{-1} = 2 \bmod 13$, so $x = 2 \times 5 = 10$.
M16	$\gcd(6, 15) = 3$, so products of 6 cannot be congruent to 1 modulo 15.

11.4 Polynomial solutions

Exercises P1-P16

No.	Answer and short path
P1	[4,0,3].
P2	$2-x+5x^3$.
P3	[5,6,4].
P4	[5,-2,7].
P5	[5,-10,15].
P6	[18,12,6] mod7=[4,5,6].
P7	[15,13,11] mod11=[4,2,0].
P8	[2-8,3-9,4-10]=[-6,-6,-6] mod13=[7,7,7].
P9	[3,10,8].
P10	[2,6,1,3].
P11	Raw [20,38,12] mod7=[6,3,5].
P12	[1,2,3,2,1].
P13	[2-5,3,4]=[-3,3,4].
P14	x^4 term4 wraps to -4 at index0; x^5 term5 to -5 at index1 -> [-3,-5,2,3].
P15	Convolution [2,5,4,1]; wrap x^3 coefficient1 -> [1,5,4].
P16	Convolution [12,26,14,2] -> mod7 [5,5,0,2]; wrap index3 subtract2 from index0 -> [3,5,0].

11.5 Vector and matrix solutions

Exercises V1-V12

No.	Answer and short path
V1	$2 \times 4 + 3 \times 5 = 23$.
V2	$3 - 10 + 8 = 1$.
V3	$0 + 12 + 7 + 6 = 25$.
V4	$15 + 8 + 6 = 29 \bmod 7 = 1$.
V5	$[1 \times 5 + 3 \times 2, 2 \times 5 + 4 \times 2] = [11, 18]$.
V6	$[8 + 0 + 6, 12 + 5 - 6] = [14, 11]$.
V7	$[11, 14, 12]$.
V8	$[8 + 6 + 1, 2 + 15 + 3] = [15, 20]$.
V9	$[12 + 3, 24 + 5] = [15, 29] \bmod 7 = [1, 1]$.
V10	$[21 + 6 + 8, 6 + 48 + 10] = [35, 64] \bmod 11 = [2, 9]$.
V11	$[40 + 54, 96 + 24, 24 + 42] = [94, 120, 66] \bmod 13 = [3, 3, 1]$.
V12	$A \cdot s = [4, 11]$; add $e = [5, 10]$; $\bmod 7 = [5, 3]$.

11.6 Rounding and noise solutions

Exercises R1-R16

No.	Answer and short path
R1	$8-11=-3$.
R2	$12-17=-5$.
R3	$23/6=3.833 \rightarrow 4$.
R4	$29/8=3.625 \rightarrow 4$.
R5	$\text{round}(8 \times 2/17) = \text{round}(0.941) = 1$.
R6	$48/17=2.824 \rightarrow 3$.
R7	$88/17=5.176 \rightarrow 5$.
R8	$120/17=7.059 \rightarrow 7$.
R9	$17/8=2.125 \rightarrow 2$.
R10	$51/8=6.375 \rightarrow 6$.
R11	$85/8=10.625 \rightarrow 11$.
R12	$102/8=12.75 \rightarrow 13$.
R13	$0+3=3$, closer to 0 than 8 $\rightarrow$ bit 0.
R14	$8-2=6$, closer to 8 $\rightarrow$ bit 1.
R15	$0+5=5$, distance to 8 is 3 and to 0 is 5 $\rightarrow$ decoder would choose bit 1; noise crossed the toy boundary.
R16	It reduces bandwidth/storage while preserving enough coarse information for decoding.

11.7 State, sponge, and stream solutions

Exercises S1-S12

No.	Answer and short path
S1	$[4,2,7,1 \text{ XOR } 4=5] \rightarrow [4,2,7,5]$.
S2	$F \text{ XOR } 3=C \rightarrow [3,8,1,C]$.
S3	First $[5,6,1,7]$; second $[6,1,7,3]$.
S4	Old a and old b.
S5	Absorb $[2,5]$: $[2,5 0,0]$; $P \rightarrow [5,0 0,7]$.
S6	Rate XOR $[1,7] \rightarrow [4,7 0,7]$; $P \rightarrow [7,0 7,3]$.
S7	First squeezed rate $[7,0]$.
S8	Rate is the state portion directly XORed/read; capacity is the internal portion not directly exposed in the basic sponge interface.
S9	$56 \text{ XOR } B1 = E7$.
S10	$E7 \text{ XOR } B1 = 56$.
S11	$[A5,F0,F5]$.
S12	$(P \text{ XOR } K) \text{ XOR } K = P \text{ XOR } (K \text{ XOR } K) = P \text{ XOR } 0 = P$.

11.8 Nim-reading solutions

Exercises N1-N12

No.	Answer and short path
N1	8 elements, each 32 bits; valid indexes 0..7.
N2	[1,2,9,4].
N3	var may be reassigned; let is bound once after initialization.
N4	15.
N5	$i=0,1,2,3 \rightarrow [0,2,4,6]$.
N6	Start1, then2,4,8 $\rightarrow$ 8.
N7	256 iterations.
N8	A secret-dependent stopping point can expose information through runtime or observable work count.
N9	$270 \bmod 256 = 14$.
N10	$-2 \bmod 256 = 254$.
N11	Two obvious temporary copies remain.
N12	Other copies, CPU registers, allocator memory, swap, crash dumps, compiler temporaries, or backups/snapshots are possible examples.

11.9 Cumulative solutions

Pipelines C1-C5

No.	Answer and short path
C1	$x=3C=001111100$; $k=A7=10100111$. $XOR=10011011=9B$. $Rotl3=11011100=DC$. $AND\ F7\ (11110111) \rightarrow 11010100=D4=212$.
C2	$x=11001001$. $NOT=00110110$. $OR00000101=00110111$. $Rotr2=11001101$. $XOR\ original=00000100=0x04$.
C3	$A*s=[17,30]$. Add $e \rightarrow [16,32]$. $Mod17 \rightarrow b=[16,15]$. Compress: $x16 \rightarrow 0$; $x15 \rightarrow 7$. Decompress $\rightarrow [0,15]$. Centered differences: $0-16=-16 \equiv 1$; $15-15=0$.
C4	Convolution $[6,14,11,11,2]$. Reduce x^3+1 : $[6-11,14-2,11]=[-5,12,11]$. Add $e \rightarrow [-4,11,11]$. $Mod7 \rightarrow [3,4,4]$, polynomial $3+4x+4x^2$.
C5	First absorb $[A,3]$: $[A,3 0,0] \rightarrow P=[3,0 0,9]$. Second XOR $[5,C] \rightarrow [6,C 0,9] \rightarrow P=[C,0 9,A]$. First output $[C,0]$. $P \rightarrow [0,9 A,C]$, second $[0,9]$. Nibbles form keystream bytes $C0$ and 09 . Plain $[42,19]$ XOR $[C0,09] \rightarrow [82,10]$.

12 What follows in the algorithm volumes

Vol.	Topic	Calculation focus
1	ChaCha20 and XChaCha20	quarter-round, state construction, HChaCha20, counters, AEAD connection
2	Poly1305 and ChaCha20-Poly1305	one-time key, block accumulation, modular reduction, tag verification
3	SHA3 and SHAKE	Keccak state, theta/rho/pi/chi/iota, absorb, padding, squeeze
4	BLAKE3	compression function, message schedule, chunk tree, keyed and derive-key modes
5	AES-CTR	byte substitution, row/column transforms, key schedule, counters
6	Gimli and generic sponges	permutation rounds, domains, tag and stream modes
7	Argon2 and the custom KDF	memory blocks, indexing, passes, lanes, lifetimes
8	X25519	field limbs, Montgomery ladder, inversion, implementation passes
9	Ed25519	point arithmetic, scalar processing, hashing, signing and verification
10	Kyber / ML-KEM	polynomial arithmetic, NTT, encryption core, FO transform, decapsulation
11	Dilithium / ML-DSA	NTT, decomposition, hints, rejection loops, sign and verify
12	SABER	Module-LWR, schoolbook multiplication, rounding, FO transform
13	NTRU	ring inverses, sampling, product-form calculations, encode/decode
14	FrodoKEM	large LWE matrices, streamed generation, noise, encode/decode
15	BIKE	binary polynomial multiplication, syndrome, black-gray decoding
16	Classic McEliece	finite fields, Goppa codes, syndrome decoding, key generation
17	Falcon	NTRU equation, FFT representation, Gaussian sampling, sign/verify
18	SPHINCS+	WOTS+, FORS, Merkle trees, hypertree signatures
19	Nim, RAM, SIMD, and optimization	buffers, cache, stack/heap, vector lanes, benchmark interpretation
20	Security engineering	constant-time limits, side channels, key lifetimes, randomness, validation
21	Mathematical reference	only the theorems and proofs needed to justify earlier calculations

13 Primary references and repository links

Reference	Link
Tyr-Crypto repository	https://github.com/siriuslee69/Tyr-Crypto
Algorithm inventory	https://github.com/siriuslee69/Tyr-Crypto/blob/main/docs/ALGORITHMS.md
Code layout	https://github.com/siriuslee69/Tyr-Crypto/blob/main/docs/CODE_LAYOUT.md
Research index: PQ families	https://github.com/siriuslee69/Tyr-Crypto/tree/main/docs/research/pq_non_ntru_saber
Research index: NTRU and SABER	https://github.com/siriuslee69/Tyr-Crypto/tree/main/docs/research/ntru_saber
ChaCha20-Poly1305 RFC 8439	https://www.rfc-editor.org/rfc/rfc8439
XChaCha construction draft	https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-xchacha
FIPS 202 SHA-3 standard	https://csrc.nist.gov/pubs/fips/202/final
BLAKE3 paper/specification	https://github.com/BLAKE3-team/BLAKE3-specs
Argon2 RFC 9106	https://www.rfc-editor.org/rfc/rfc9106
X25519 RFC 7748	https://www.rfc-editor.org/rfc/rfc7748
Ed25519 RFC 8032	https://www.rfc-editor.org/rfc/rfc8032
ML-KEM FIPS 203	https://csrc.nist.gov/pubs/fips/203/final
ML-DSA FIPS 204	https://csrc.nist.gov/pubs/fips/204/final
SLH-DSA FIPS 205	https://csrc.nist.gov/pubs/fips/205/final
CRYSTALS-Kyber paper	https://eprint.iacr.org/2017/634
CRYSTALS-Dilithium paper	https://eprint.iacr.org/2017/633
Frodo paper	https://eprint.iacr.org/2016/659
Falcon specification	https://falcon-sign.info/falcon.pdf
BIKE site/specification	https://bikesuite.org/
Classic McEliece specification	https://classic.mceliece.org/
NTRU specification site	https://ntru.org/
SABER specification site	https://www.esat.kuleuven.be/cosic/pqcrypto/saber/

Safety boundary

This handbook explains calculations and implementation structure. It does not certify the repository for production use. Constant-time behavior, compiler transformations, platform details, protocol composition, randomness, key management, and independent review all matter.